

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Design a banking system in Java with:

A base class Account containing private fields: accountNumber, ownerName, balance

Provide getters/setters and at least two constructors (use constructor chaining)

Implement deposit() and withdraw() with proper validation

Add a display() method

Extend it with:

SavingsAccount (add interestRate, override display() and show interest)

CurrentAccount (add overdraftLimit, restrict withdrawals accordingly)

Your implementation should clearly show:

Proper encapsulation (no direct field access)

Use of constructor overloading and chaining (this(...)) Inheritance and method overriding (use @Override and super) Polymorphism by storing objects in an Account reference list and calling display() Basic validation/debugging (e.g., assert or exception for invalid operations)

Answer:

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
class Account {  
    private String accountNumber;  
    private String ownerName;  
    private double balance;
```

```
public Account() {  
    this("0000", "Unknown", 0.0);  
}
```

```
public Account(String accountNumber, String ownerName, double balance) {  
    this.accountNumber = accountNumber;  
    this.ownerName = ownerName;  
    setBalance(balance);  
}
```

```
public String getAccountNumber() {  
    return accountNumber;  
}
```

```
public void setAccountNumber(String accountNumber) {  
    this.accountNumber = accountNumber;  
}
```

```
public String getOwnerName() {  
    return ownerName;  
}
```

```
public void setOwnerName(String ownerName) {  
    this.ownerName = ownerName;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

```
public void setBalance(double balance) {
```

```
    if (balance < 0) {  
        throw new IllegalArgumentException("Balance cannot be negative");  
    }  
    this.balance = balance;  
}
```

```
protected void updateBalance(double balance) {  
    this.balance = balance;  
}
```

```
public void deposit(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("Invalid deposit amount");  
    }  
    balance += amount;  
}
```

```
public void withdraw(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("Invalid withdrawal amount");  
    }  
}
```

```
    if (amount > balance) {  
        throw new IllegalArgumentException("Insufficient balance");  
    }  
  
    balance -= amount;  
}
```

```
public void display() {  
    System.out.println("Account Number: " + accountNumber);  
}
```

```
        System.out.println("Owner Name: " + ownerName);
        System.out.println("Balance: " + balance);
    }
}
```

```
class SavingsAccount extends Account {
    private double interestRate;

    public SavingsAccount() {
        this("0000", "Unknown", 0.0, 0.0);
    }

    public SavingsAccount(String accountNumber, String ownerName, double
balance, double interestRate) {
        super(accountNumber, ownerName, balance);
        this.interestRate = interestRate;
    }

    public double calculateInterest() {
        return getBalance() * interestRate / 100;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Interest Rate: " + interestRate + "%");
        System.out.println("Interest: " + calculateInterest());
        System.out.println();
    }
}
```

```
class CurrentAccount extends Account {
    private double overdraftLimit;

    public CurrentAccount() {
        this("0000", "Unknown", 0.0, 0.0);
    }

    public CurrentAccount(String accountNumber, String ownerName, double
balance, double overdraftLimit) {
        super(accountNumber, ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid withdrawal amount");
        }

        if (amount > getBalance() + overdraftLimit) {
            throw new IllegalArgumentException("Overdraft limit exceeded");
        }

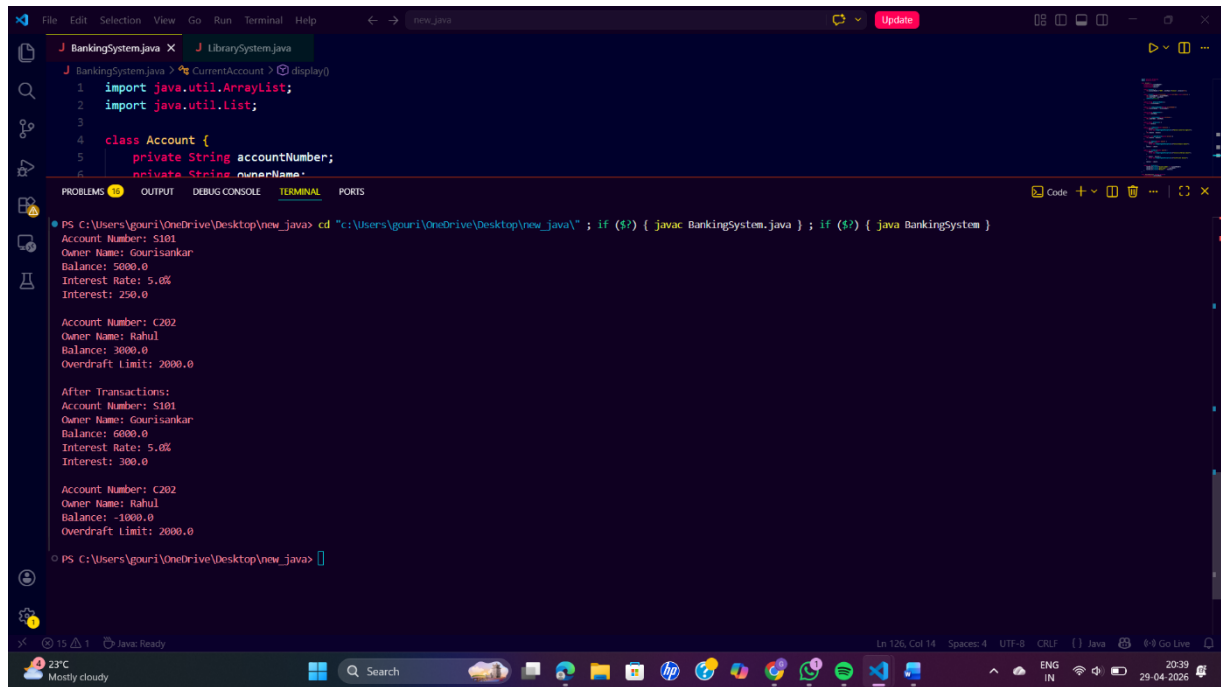
        updateBalance(getBalance() - amount);
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Overdraft Limit: " + overdraftLimit);
        System.out.println();
    }
}
```

```
}  
}
```

```
public class BankingSystem {  
    public static void main(String[] args) {  
        List<Account> accounts = new ArrayList<>();  
  
        accounts.add(new SavingsAccount("S101", "Gourisankar", 5000, 5));  
        accounts.add(new CurrentAccount("C202", "Rahul", 3000, 2000));  
  
        for (Account acc : accounts) {  
            acc.display();  
        }  
  
        accounts.get(0).deposit(1000);  
        accounts.get(1).withdraw(4000);  
  
        System.out.println("After Transactions:");  
  
        for (Account acc : accounts) {  
            acc.display();  
        }  
    }  
}
```

Screenshots of the output:



The screenshot shows an IDE with two tabs: `BankingSystem.java` and `LibrarySystem.java`. The `BankingSystem.java` tab is active, displaying the following code:

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Account {
5     private String accountNumber;
6     private String ownerName;
```

The terminal output shows the execution of the program, displaying account details for two accounts and the results of transactions.

```
PS C:\Users\gauri\OneDrive\Desktop\new_java> cd "c:\Users\gauri\OneDrive\Desktop\new_java\" ; if ($?) { javac BankingSystem.java } ; if ($?) { java BankingSystem }
Account Number: S101
Owner Name: Gourisankar
Balance: 5000.0
Interest Rate: 5.0%
Interest: 250.0

Account Number: C202
Owner Name: Rahul
Balance: 3000.0
Overdraft Limit: 2000.0

After Transactions:
Account Number: S101
Owner Name: Gourisankar
Balance: 5800.0
Interest Rate: 5.0%
Interest: 300.0

Account Number: C202
Owner Name: Rahul
Balance: -1000.0
Overdraft Limit: 2000.0

PS C:\Users\gauri\OneDrive\Desktop\new_java>
```

The IDE interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help), a toolbar with icons for file operations and running code, and a status bar at the bottom showing the current file path, line and column numbers, and the Java version.